National University of Computer and Emerging Sciences



Laboratory Manuals

for

Computer Networks - Lab

(CL -3001)

Course Instructor	Dr. Abdul Qadeer
Lab Instructor	Ms. Halima Sadia
Section	BCS-5A
Semester	Fall 2026

Department of Computer Science
FAST-NU, Lahore, Pakistan

Lab Manual 06

CLOs

CLO 2: Demonstrate an understanding of the fundamental concepts of socket programming, types of protocols, and IP addressing.

Objective

- By the end of this lab, students should be able to:
- Implement a TCP server that handles multiple clients using threads.
- Understand how a multithreaded server handles multiple client connections.
- Authenticate clients using information stored in a text file.
- Process client requests concurrently.
- Synchronize access to shared voting records.

Prerequisites

Students should have basic knowledge of:

- Basic threading concepts
- Client-server communication.
- TCP and transport-layer protocols.
- Basic IP addressing.

Required files:

The following provided files:

1. TCP_Client.py
2. TCP_Server.py
3. Voters_List.txt
4. Candidates_List.txt

TCP Multithreading

A TCP server normally accepts connections from clients and communicates with them through sockets. A multithreaded TCP server creates a separate thread to handle each connected client.

This allows the server to handle multiple clients without waiting for one client to completely finish before accepting another.

In this lab, the server will use TCP and Python threads to handle multiple voters.

Lab Statement: Online Voting System

Total Marks: 20

Implement a simulated Online Voting System using a TCP multithreaded client-server architecture.

Multiple clients will connect to the server. Each client represents a registered voter.

The system must:

1. Authenticate the voter using their Name and CNIC.
2. Allow only authenticated voters to proceed.
3. Display the available candidates and their poll symbols.
4. Allow the authenticated voter to cast a vote by entering the poll symbol, not the candidate name.
5. Ensure that each registered voter can vote once and only once.
6. Handle multiple clients concurrently using threads.
7. Record the voting information in an output text file.

1. Required Input Files

The following files are provided with the lab.

Voters_List.txt

This file contains the registered voters in the following format:

```
Name/CNIC
```

Example:

```
Ali/35202-1234567-1
Sara/35202-7654321-2
Ahmed/35202-1111111-3
```

Each line represents one registered voter.

Candidates_List.txt

This file contains the available candidates and their poll symbols. Use the following format:

```
Candidate Name/Poll Symbol
```

Example:

```
Candidate A/AA
Candidate B/BB
Candidate C/CC
```

The candidate information in the actual file provided with the lab must be used by the program.

2. Client-Server Workflow

For each client:

1. The client establishes a TCP connection with the server.
2. The server requests the voter's Name.
3. The client sends the name.
4. The server requests the voter's CNIC.
5. The client sends the CNIC.
6. The server checks the supplied information against Voters_List.txt.
7. If the credentials are not found, the server rejects the request and closes the client connection.
8. If the credentials are valid, the server welcomes the voter.
9. The server sends the candidate names and their corresponding poll symbols to the client.
10. The client selects a candidate by entering the poll symbol.
11. The server validates the poll symbol.
12. If the voter has not voted previously, the vote is recorded.
13. The server confirms that the vote has been successfully recorded.
14. The client connection is closed after voting is completed.
15. The server continues running and remains available for other clients.

3. Voter Authentication

The server must authenticate each voter using both Name and CNIC.

The supplied Name and CNIC must match a record in:

```
Voters_List.txt
```

Authentication Success

If both details match a registered voter:

```
Welcome, Ali.
```

The voter may proceed to the voting stage.

Authentication Failure

If the Name/CNIC combination does not match a registered voter:

```
Authentication failed. You are not a registered voter.
```

The voter must not be allowed to cast a vote.

4. One Vote Per Voter

Each authenticated voter can cast one and only one vote.

The server must maintain a record of voters who have already voted.

If a voter who has already voted attempts to vote again, the server must reject the request.

Example:

```
You have already cast your vote. You cannot vote again.
```

The server must not record another vote for that voter.

5. Candidate Information

After successful authentication, the server must provide the candidate information from:

```
Candidates_List.txt
```

The client should receive and display:

```
Candidate A - AA
Candidate B - BB
Candidate C - CC
```

The voter must select a candidate using the poll symbol. For example:

```
Enter poll symbol: BB
```

The voter must not select the candidate by entering the candidate's name.

6. Vote Validation

The server must verify that the poll symbol entered by the voter exists in Candidates_List.txt.

Valid Poll Symbol

If the symbol is valid, the server records the vote. Example:

```
Vote recorded successfully.
```

Invalid Poll Symbol

If the symbol does not match any candidate:

```
Invalid poll symbol. Please enter a valid poll symbol.
```

The invalid vote must not be recorded.

7. Output File

All successfully cast votes must be recorded in an output text file. Use:

```
Votes.txt
```

Each record should contain the voter's identity and the selected poll symbol. Use the following format:

```
Name/CNIC/Poll Symbol
```

Example:

```
Ali/35202-1234567-1/AA
Sara/35202-7654321-2/BB
```

Do not record a voter more than once.

8. Multithreading Requirement

The server must create a separate thread for each connected client.

The main server thread should:

1. Create the TCP socket.
2. Bind the socket to the server IP and port.
3. Listen for incoming connections.
4. Accept a client connection.
5. Create a new thread to handle that client.
6. Immediately continue listening for other clients.

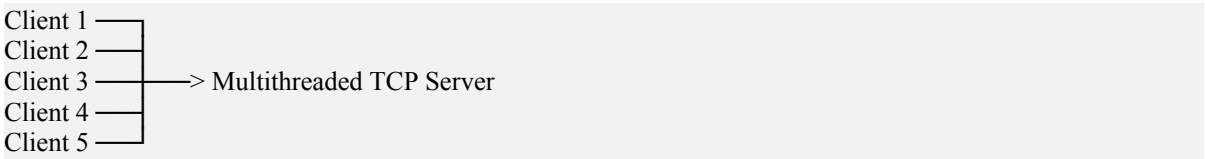
The client-handling thread should perform the authentication and voting process for its assigned client.

The server must not handle all clients sequentially in a single thread.

9. Concurrent Client Requirement

The system must support at least 5 different clients.

The server should be able to accept connections from multiple clients without being restarted. For example:



Each client should be handled by its own server thread.

10. Shared Voting Records

The list or data structure containing voters who have already voted is shared between client-handling threads.

Therefore, access to the shared voting records must be synchronized.

Use an appropriate Python synchronization mechanism, such as:

```
threading.Lock()
```

The lock should protect the operation that checks whether a voter has already voted and records the vote.

This prevents two concurrent requests from causing the same voter to be recorded more than once.

11. Required Test Cases

Test the system using the following cases.

Test Case	Input/Action	Expected Result
Valid Voter	Registered Name + CNIC	Authentication succeeds
Invalid Voter	Name/CNIC not in Voters_List.txt	Authentication rejected
Valid Vote	Valid poll symbol	Vote recorded
Invalid Symbol	Symbol not in Candidates_List.txt	Vote rejected
Duplicate Vote	Same authenticated voter attempts to vote again	Vote rejected
Multiple Clients	At least 5 clients connect	Server handles clients using separate threads
Concurrent Access	Multiple clients vote while server is running	Votes are recorded correctly without duplicate records

12. Sample Test Bench

Assume the files contain:

Voters_List.txt

```
Ali/35202-1234567-1
Sara/35202-7654321-2
Ahmed/35202-1111111-3
```

Candidates_List.txt

```
Candidate A/AA
Candidate B/BB
Candidate C/CC
```

Client 1

```
Name: Ali
CNIC: 35202-1234567-1
```

Server:

```
Authentication successful.
Welcome, Ali.

Available Candidates:
Candidate A - AA
Candidate B - BB
Candidate C - CC
```

Client:

```
Enter poll symbol: AA
```

Server:

```
Vote recorded successfully.
```

Votes.txt becomes:

```
Ali/35202-1234567-1/AA
```

13. Duplicate Vote Test

If Ali attempts to vote again:

```
Name: Ali
CNIC: 35202-1234567-1
```

The server should respond:

```
You have already cast your vote. You cannot vote again.
```

The server must not add another record for Ali to Votes.txt.

14. Server Continuity

The server must remain running after a client completes the voting process. For example:

```
Client 1 connects
   ↓
Client 1 votes
   ↓
Client 1 disconnects
   ↓
Server continues running
   ↓
Client 2 connects
   ↓
Client 2 votes
   ↓
Server continues running
```

The server should only terminate when intentionally stopped.

15. Submission Requirements

Submit:

1. TCP_Client.py
2. TCP_Server.py
3. Voters_List.txt
4. Candidates_List.txt
5. Votes.txt
6. Required screenshots

Required Screenshots

Your screenshots must demonstrate:

1. Successful voter authentication.
2. Candidate list displayed with poll symbols.
3. Successful vote casting.
4. Invalid voter authentication.

5. Invalid poll symbol handling.
6. Duplicate vote rejection.
7. Multiple clients being handled by the server.
8. Server remaining active while handling multiple clients.
9. Votes.txt containing successfully recorded votes.

16. Marks Distribution

Component	Marks
TCP client-server communication	3
Voter authentication using Voters_List.txt	3
Candidate list and poll-symbol processing	3
One-vote-only enforcement	3
Multithreaded server implementation	4
Vote recording and file handling	2
Testing and screenshots	2
Total	**20**

Important Notes

- Use TCP for client-server communication.
- Use a separate thread for each connected client.
- The server must remain running while clients connect and vote.
- Only voters whose Name and CNIC match a record in Voters_List.txt may vote.
- Each registered voter may cast one vote only.
- The voter must select a candidate using the poll symbol, not the candidate name.
- The server must validate the poll symbol before recording the vote.
- Shared voting records must be accessed safely when multiple client threads are running.
- Do not modify the supplied format of Voters_List.txt and Candidates_List.txt.
- Use only the fictional/sample data provided for this lab; this is a programming exercise, not a real-world voting system.